

Written Analysis

Question 1

$h(n)$ = wrong_ones (the number of positions that are not the same as the goal)

Steps

- $h(n)$ = #positions where state is not equal to goal
- Lower bounds: the least actions one would need, since every action can only fix one letter position at a time. So if 2 letters are wrong, you need at least 2 more moves
- It cannot overestimate because no move can ever cost less than 1 point, so fixing wrong_ones letters costs at least wrong_ones. Therefore, the actual cost must be equal or higher.

Scrabble

- $h(n)$ = wrong_ones
- Lower bounds: uses the cheapest cost under the impression that every letter will use the lower letter where cost = 1.
- Cannot overestimate because cost = 1 is the lowest cost value; there is no other cost lower than that. The cost can be equal to or higher only.

Frequency

- $h(n)$ = wrong_ones
- Lower bounds: cheapest total cost, under the impression that every move gets the word with 0 rarity where cost is $1+0 = 1$.
- Cannot overestimate because rarity is not negative, so cost is at least 1. Therefore, the cost has to be at least wrong_ones.

Question 2

Word pair	Cost measure	UCS cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
WHY->ASK	steps	5	773	5	16	97.9%
WHY->ASK	scrabble	9	765	9	353	53.9%
WHY->ASK	frequency	38.86	808	38.86	755	6.6%
BEAR->DUCK	steps	4	691	4	6	99.1%
BEAR->DUCK	scrabble	11	2981	11	1376	53.9%
BEAR->DUCK	frequency	38.24	1084	38.24	720	33.6%

- 1) Yes, my A* and UCS did return the same optimal cost
- 2) Both pairs; steps had it higher because it reached 97% to 99% while it was under 50s for scrabble and frequency

- 3) Both pairs; frequency had the least about 6.6% reduction for WHY to ASK and 33.6% on BEAR to DUCK which is way lower than scrabble and steps.
- 4) - steps: since each action only costs about 1, wrong_ones is close to the actual remaining cost. Therefore, A* prunes almost all that UCS would.
 - scrabble: letters can cost anywhere from 1 to 10 points. My heuristic always assumes the cheapest letter (cost 1), even when the actual path needs an expensive one like k, j, x, q, or z. So it guesses way too low a lot of the time, and that bad guess is why A* doesn't skip many extra nodes compared to UCS.
 - frequency: word rarity values go from almost 0 all the way past 16. My heuristic assumes every word will be as common as possible (rarity 0), which is almost never true. Since the guess is so far off from the real cost, A* barely narrows things down. Therefore, it ends up expanding nearly as many nodes as UCS.

Question 3

Steps: Yes, same cost both ways. Every move costs 1, period. Doesn't matter what letter you're putting in. So if it takes 5 moves to go from W1 to W2, going back the same way also takes 5 moves. Same cost.

Scrabble: No, not the same. The cost depends on the letter you're adding, not the letter you're taking out. So going one way you pay for one letter, going back you pay for a different letter which can cost different amounts. Ex: dog to dot costs 1 (you added 't'). But dot to dog costs 2 (you added 'g'). Same swap, different price depending on direction.

Frequency: No, not the same either. The cost depends on how rare the word you land on is, not the word you left. So going one way you pay based on one word's rarity, coming back you pay based on the other word's rarity. Those numbers are almost never equal. Ex: same dog/dot pair. Going dog to dot, you pay based on how rare "dot" is. Going dot to dog, you pay based on how rare "dog" is. Different words, different rarity, different cost.

Question 4

- 1) GPT-5.6 Luna. I pasted the exact prompt from the assignment. The model's first idea wasn't safely admissible by its own admission, so it proposed a safer fallback: $h_R(n) = h_0(n) + L(n)$, where $L(n)$ is some rarity cost that's unavoidable no matter what path you take.
- 2) $r = 0.016226212$
 $h_R(n) = \text{wrong_ones}(n) + r$
- 3) Yes. Every single action you take has to land on some word in the dictionary, and no word in the dictionary has rarity below r . So every action costs at least $1 + r$. That means the true remaining cost is at least $\text{wrong_ones} * (1 + r)$, which is definitely greater than or equal to $\text{wrong_ones} + r$ (since wrong_ones greater than or equal to 1 whenever you're not at the goal). So this heuristic never overestimates making it safe.

- 4) The AI's own first idea wasn't safe, but its fallback is already the fixed version, so no further repair needed.
- 5) Both ran the exact same number of nodes (755 for why to ask), 720 for bear to duck, same cost too. No difference because adding r to every h value doesn't actually change anything. A^* just cares about which node has the lowest score, and if you add the same amount to every single one, the order they get picked in stays exactly the same. Kinda like everyone getting +5 points on a test. Nobody's rank changes. Therefore, it technically checks the boxes (admissible, slightly bigger than h_0), but it does nothing in practice. I wouldn't use it over the regular `wrong_ones` heuristic. It's just extra math for no real gain.

I used Claude (Anthropic) for implementing a few methods in `dc.py` (`actions`, `result`, `path_cost`, `h`) and verifying/testing the Q4 heuristic. I reviewed the code and the reasoning it gave me and ran all the tests myself to confirm the results.